

Assignment : RE605 - PATH PLANNING

Name: [4222301042] Aldon Zufar Putra Twyn

Class : Robotika – B Pagi [Semester 5]

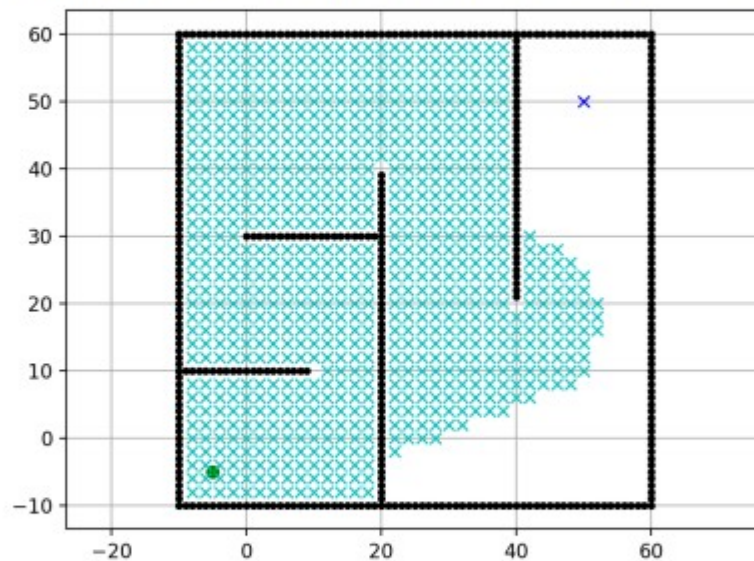
Topic : Mempelajari Algoritma Dijkstra dan D* [Practice #3 & #4]

QUESTIONS PRACTICE 2 & 3

4.4 Pertanyaan Diskusi

Untuk mengukur pemahaman setelah membaca materi diatas, maka perlu dilakukan asesmen formatif dengan cara menjawab pertanyaan berikut:

1. Bagian program manakah yang perlu diubah jika ada penambahan obstacle seperti pada gambar dibawah ini?



Gambar 4-6 Penambahan Obstacle

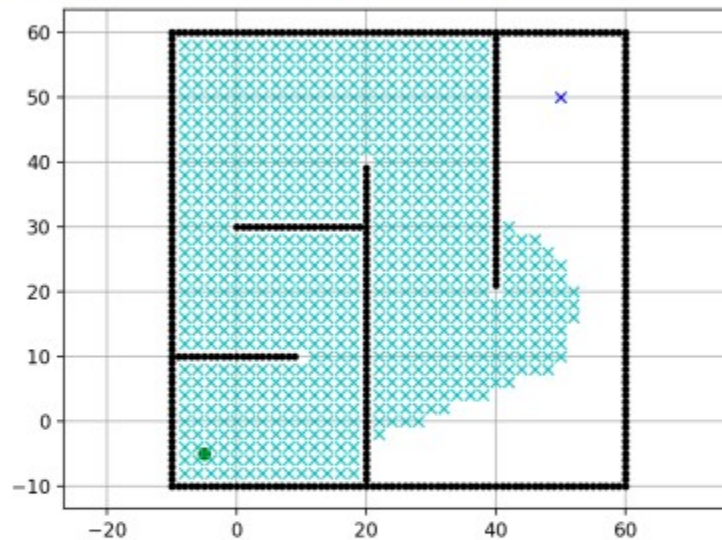
2. Dengan peta obstacle yang sama seperti Gambar 4-6, buatlah simulasi algoritma Dijkstra's dan algoritma A*!
3. Jelaskan perbedaan antara algoritma Dijkstra's dan A*?

Soal pertanyaan untuk Practice #2 [Dijkstra]

5.6 Pertanyaan Diskusi

Untuk mengukur pemahaman setelah membaca materi diatas, maka perlu dilakukan asesmen formatif dengan cara menjawab pertanyaan berikut:

1. Dengan peta obstacle yang sama seperti Gambar 5-3, buatlah simulasi algoritma Dijkstra's, A* dan D*!

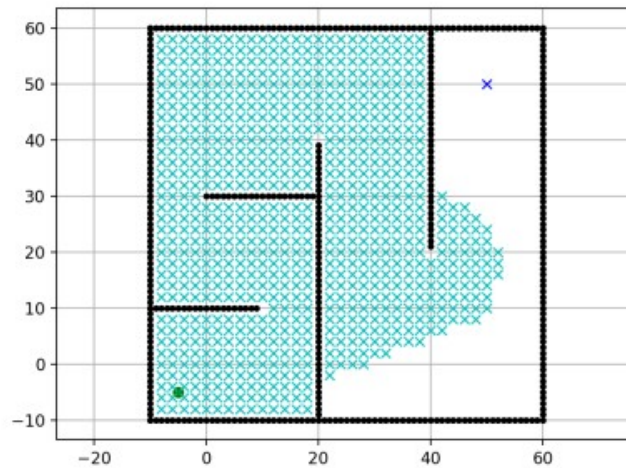


Gambar 5-3 Peta Obstacle

2. Jelaskan perbedaan antara algoritma Dijkstra's, A* dan D*?

Soal pertanyaan untuk Practice #3 [D]*

Untuk menjawab pertanyaan “Bagian program manakah yang perlu diubah jika ada penambahan obstacle seperti pada gambar dibawah ini?”



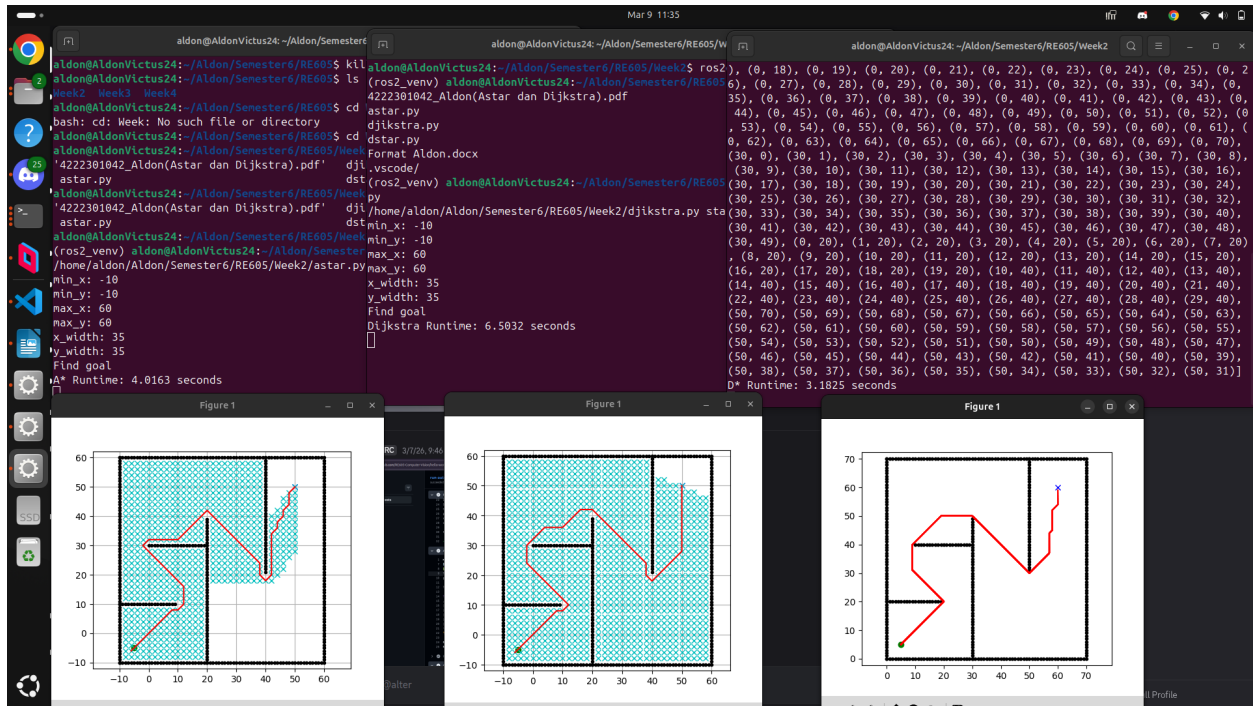
Gambar 4-6 Penambahan Obstacle

Jawabannya adalah berikut:

```
File Edit Selection View Go Run Terminal Help
+ astar.py + djikstra.py x + dstar.py
Week2 > djikstra.py > main
100
161 def main():
162     print(__file__ + " start!!")
163
164     sx = -5.0
165     sy = -5.0
166     gx = 50.0
167     gy = 50.0
168     grid_size = 2.0
169     robot_radius = 1.0
170
171     ox, oy = [], []
172     for i in range(-10, 60):
173         ox.append(i)
174         oy.append(-10.0)
175     for i in range(-10, 60):
176         ox.append(60.0)
177         oy.append(i)
178     for i in range(-10, 61):
179         ox.append(i)
180         oy.append(60.0)
181     for i in range(-10, 61):
182         ox.append(-10.0)
183         oy.append(i)
184     for i in range(-10, 40):
185         ox.append(20.0)
186         oy.append(i)
187     for i in range(0, 40):
188         ox.append(40.0)
189         oy.append(60.0 - i)
190
191     for i in range(0, 20):
192         ox.append(i)
193         oy.append(30.0)
194     for i in range(-10, 10):
195         ox.append(i)
196         oy.append(10.0)
```

Yang perlu di ubah terletak pada baris kisaran 170-190an untuk kedua kode (A* dan Dijkstra) yang dimana disini kita dapat menambahkan dinding area dengan menyatakan letak koordinatnya dan ukurannya dari ujung ke ujung dinding sesuai preferensi sendiri.

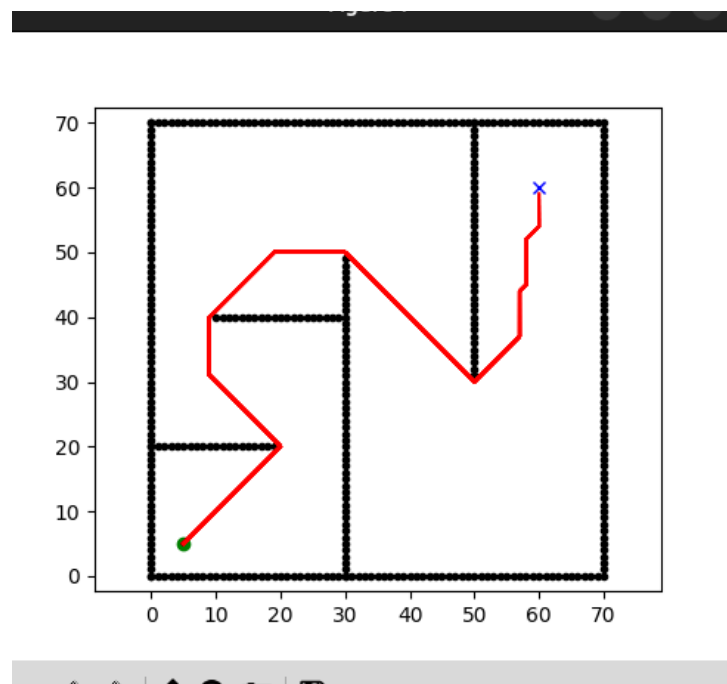
PRACTICE - #3



Sama seperti percobaan practice - #3, disini saya juga sudah memperbaiki dan menjalankan program D* yang sudah disediakan di E-Learning dan saya memodifikasinya sedikit dengan menambahkan Timer dan juga memperbaiki layout arena agar semua program berada pada arena yang sama.

Disini saya membandingkan ketiga program yang dimana dari gambar kiri ke kanan urutannya sebagai berikut yaitu: **A***, **Dijkstra**, **D***.

Untuk Kecepatan D* dan A* jauh lebih unggul daripada Dijkstra namun perlu diketahui A* dan D* memerlukan letak lokasi goal sedangkan Dijkstra dapat dijalankan di environment yang dimana letak goal belum diketahui.



Dapat dilihat juga program D* saya di sebelah ini, garis path atau garis merah perjalanan terlihat pergerakannya lebih efisien dan lebih baik ketimbang 2 program yang di atas, ini karena D* itu seperti melakukan "Traceback" atau tempuh balik solah2 kita sudah melewati suatu jalan lalu kita ingin kembali pulang dan kita sudah gampang melewati jalan karena kita sudah menempuh perjalanan itu dan dapat memetakan route mana yng terbaik untuk kembali pulang.

Apa saja perbedaan antara A*, Dijkstra dan D*?

Ringkas saja berikut adalah perbedaannya:

Dijkstra = Menjelajah setiap tempat (node) secara buta yang diperlukan untuk mengetahui layout tempat hingga mencapai titik goal (**Lambat, Buta, namun Cocok untuk lingkungan dimana goal tidak diketahui**)

A* = Menggunakan Heuristic dan mengestimasi jarak perkiraan dari titik mulai ke titik Goal/Finish agar tidak memerlukan banyak komputasi dan dapat memetakan jalur tercepat (**Cepat, singkat, path lumayan optimal namun memerlukan lokasi titik goal dan perlu pathing lagi jika ada perubahan map**)

D* = Merupakan A* namun dengan pencarian arah direksi yang dibalik. D* lebih dinamis terutama jika ada gangguan pada area karena melainkan A* yang perlu pathing/planning ulang ketika ada perubahan pada arena, D* hanya perlu menghitung ulang jarak yang terpengaruh dan dapat melanjutkan haluan. (**Sama seperti A* namun dengan orientasi permulaan dari Goal-ke-Start, Dapat menangani perubahan pada environment. Kurang Lebih seperti A* yang sudah di Upgrade**)